

로봇 개발자를 위한 ①

ROS2 프로그래밍 첫걸음

(로봇 개발자를 위한 ①)

ROS2 프로그래밍 첫걸음

발 행 | 2026년 08월 08일

저 자 | 이수미, 손민호, 최규남, 이하나

펴낸이 | 한건희

펴낸곳 | 주식회사 부크크

출판사등록 | 2014.07.15.(제2014-16호)

주 소 | 서울특별시 금천구 가산디지털1로 119 SK트윈타워 A동 305호

전 화 | 1670-8316

이메일 | info@bookk.co.kr

ISBN | 979-11-410-0000-0

www.bookk.co.kr

© (로봇 개발자를 위한 ①) ROS2 프로그래밍 첫걸음

본 책은 저작자의 지적 재산으로서 무단 전재와 복제를 금합니다.

로봇 개발자를 위한 ①

ROS2 프로그래밍 첫걸음

이수미, 손민호, 최규남, 이하나 지음

CONTENT

지은이 소개	11
머리말	12
이 책의 예제 코드	14
GitHub 안내	15

제1부 ROS2 시작하기

제1장 ROS2 개요와 개발 환경 구축 18

1.1 로봇 소프트웨어와 ROS	18
1.2 ROS1에서 ROS2로	21
1.3 ROS2의 심장, DDS	23
1.4 배포판 선택 - 왜 Jazzy인가	27
1.5 다섯 가지 통신 미리보기	29
1.6 개발 환경 준비	30
1.7 ROS2 Jazzy 설치	32
1.8 환경 설정	34
1.9 설치 확인 - 첫 통신	36
1.10 [미니 실습] DDS 구현 바꿔 보기	38
1.11 자주 만나는 오류와 해결	39

제2장 워크스페이스와 패키지, 빌드 시스템 41

2.1 ROS2 워크스페이스 이해하기	42
----------------------------	----

2.2 colcon 빌드 시스템	46
2.3 패키지 만들기 - Python과 C++	50
2.4 ament 빌드 시스템과 package.xml	55
2.5 워크스페이스 오버레이	60

제2부 ROS2 핵심 통신 (Python & C++)

제3장 토픽 - 발행/구독65

3.1 토픽 통신 모델	66
3.2 퍼블리셔 만들기	70
3.3 서브스크라이버 만들기	76
3.4 C++로 구현하기	82
3.5 QoS 설정	88
3.6 ros2 topic 명령으로 살펴보기	92

제4장 서비스 - 요청/응답95

4.1 서비스 통신 모델	96
4.2 서비스 서버와 클라이언트	100
4.3 C++로 구현하기	108
4.4 ros2 service 명령으로 살펴보기	116

제5장 액션 - 목표/피드백/결과 ... 121

5.1 액션 통신 모델	122
5.2 액션 서버와 클라이언트	128
5.3 피드백과 목표 취소	138
5.4 ros2 action 명령으로 살펴보기	146

제6장 파라미터와 커스텀 인터페이스	151
6.1 파라미터 다루기	152
6.2 파라미터 선언과 콜백	158
6.3 커스텀 메시지·서비스·액션 정의	164
6.4 인터페이스 빌드와 사용	172
 제7장 런치 시스템	 179
7.1 런치란 무엇인가	180
7.2 파이썬 런치 파일 작성	186
7.3 노드 구성과 리매핑	192
7.4 파라미터와 실행 인자 전달	198
 용어집	 205
참고문헌	210

지은이 소개

이수미 — 로봇과 인공지능 분야의 입문 교재를 집필해 왔다. 어려운 기술도 처음 접하는 사람이 끝까지 따라올 수 있게 풀어 쓰는 데 관심이 많다. 『라우팅&스위칭 첫걸음』, 『게임인공지능 첫걸음』, 『생성형AI 활용 첫걸음』을 펴냈다.

손민호 · 최규남 · 이하나 — 공저자 소개는 추후 추가됩니다.

지은이 소개

이하나 —

지은이 소개

최규남 -

지은이 소개

손민호

머리말

로봇은 더 이상 연구실과 공장의 울타리 안에만 있지 않습니다. 물류창고를 누비는 운반 로봇, 식당에서 음식을 나르는 서빙 로봇, 사람과 나란히 일하는 협동로봇, 하늘을 나는 드론까지 로봇은 우리 일상으로 빠르게 들어오고 있습니다. 이 모든 로봇의 안쪽에는 센서로 세상을 인식하고, 상황을 판단하고, 모터를 움직이는 소프트웨어가 있습니다.

로봇 소프트웨어는 한 사람이 처음부터 끝까지 만들 수 없습니다. 카메라 처리, 거리 센서 해석, 위치 추정, 경로 계획, 모터 제어가 각각 하나의 전문 분야입니다. 그래서 로봇 소프트웨어는 부품을 조립하듯 여러 모듈을 연결해 만듭니다. 이때 모듈들이 서로 대화하는 공통의 약속이 ROS(Robot Operating System)입니다.

ROS1은 오랫동안 로봇 연구의 표준이었지만, 실시간 제어와 보안, 여러 대의 로봇을 다루는 산업 현장에서는 한계가 분명했습니다. ROS2는 이 한계를 넘기 위해 산업 표준 통신 미들웨어인 DDS 위에서 처음부터 다시 설계되었습니다. 오늘날 현업의 무게중심은 빠르게 ROS2로 옮겨가고 있습니다.

이 책은 두 권으로 이루어진 시리즈입니다. 1권은 노드와 토픽, 서비스, 액션, 파라미터, 런치 등 ROS2 프로그래밍의 기본을 다루고, 2권은 URDF와 Gazebo를 이용한 시뮬레이션을 다룹니다. 두 권은 이어지지만 각각 독립적으로 읽을 수 있습니다.

이 책은 세 가지 원칙으로 썼습니다. 첫째, Python과 C++을 함께 다룹니다. 둘째, 모든 예제를 Ubuntu 24.04와 ROS2 Jazzy에서 직접 검증했고 완성된 코드를 GitHub에 공개합니다. 셋째, 명령을 나열하지 않고 그

이유를 먼저 설명합니다.

이 책은 네 사람이 함께 썼습니다. 처음 ROS2를 시작하는 사람이 덜 헤매도록, 가르치고 배우며 쌓은 경험을 한 권에 모았습니다. 이 책이 로봇 소프트웨어를 시작하는 데 도움이 되기를 바랍니다.

지은이 일동

이 책의 예제 코드

이 책의 모든 예제 코드는 GitHub 저장소 `sumilee-pcu/ROS2_2026`에 정리되어 있습니다. 장별 폴더에 Python과 C++ 예제가 함께 들어 있어, 본문을 따라 하며 바로 실행해 볼 수 있습니다.

예제는 Ubuntu 24.04와 ROS2 Jazzy에서 검증했습니다. 저장소의 README에 장별 폴더 구성과 실행 방법을 정리해 두었으니, 환경을 갖춘 뒤 README의 안내를 따르면 됩니다.

GitHub 안내

저장소 주소

이 책의 예제 코드: https://github.com/sumilee-pcu/ROS2_2026

장별 예제 구성

1장 개발 환경 구축 · 2장 워크스페이스와 빌드

3장 토픽 · 4장 서비스 · 5장 액션

6장 파라미터와 커스텀 인터페이스 · 7장 런치

각 장 폴더에는 Python(rclpy)과 C++(rclcpp) 예제가 함께 들어 있습니다. 어떤 폴더가 어떤 장에 해당하는지는 저장소 README의 표에서 확인할 수 있습니다.

제1부 ROS2 시작하기

제1장 ROS2 개요와 개발 환경 구축

이 장에서 다루는 내용

- ROS와 ROS2가 로봇 소프트웨어에서 맡는 역할과 활용 분야
- ROS1과 ROS2의 차이, 그 바탕인 DDS와 ROS2의 계층 구조
- 앞으로 배울 다섯 가지 통신 방식의 개요
- 이 책이 Jazzy 배포판과 Ubuntu 24.04를 선택한 이유
- ROS2 Jazzy 설치와 환경 설정(네이티브, WSL 2)
- talker/listener 예제로 설치를 검증하고 시스템을 확인하는 방법
- DDS 구현을 바꿔 같은 코드가 동작하는지 확인하는 미니 실습

학습 목표

이 장의 학습 목표는 다음과 같다.

1. ROS가 로봇 소프트웨어에서 맡는 역할과 활용 분야를 설명할 수 있다.
2. ROS1과 ROS2의 핵심 차이를 이해하고, ROS2가 DDS 위에서 동작하는 의미를 설명할 수 있다.
3. ROS2의 계층 구조를 이해하고, 다섯 가지 통신 방식을 구분할 수 있다.
4. 이 책이 Jazzy 배포판을 선택한 이유를 LTS 개념과 함께 설명할 수 있다.
5. Ubuntu 24.04에 ROS2 Jazzy를 설치하고 환경 변수를 설정할 수 있다.
6. talker/listener 예제로 설치를 검증하고, ros2 명령으로 노드와 토픽을 확인할 수 있다.
7. RMW 개념을 이해하고 DDS 구현을 바꿔 실행할 수 있다.

1.1 로봇 소프트웨어와 ROS

로봇은 소프트웨어 덩어리

겉으로 보면 로봇은 모터와 센서, 금속 골격으로 이루어진 기계입니다. 하지만 그 기계를 로봇으로 만드는 것은 소프트웨어입니다. 카메라가 보낸 영상에서 사람을 찾아내고, 거리 센서로 벽까지의 거리를 재고, 그 정보로 어디로 갈지 결정하고, 바퀴 모터에 속도 명령을 내리는—이 모든 일이 소프트웨어의 몫입니다.

문제는 이 일들이 저마다 다른 전문 분야라는 점입니다. 영상 인식, 센서 처리, 위치 추정, 경로 계획, 모터 제어를 한 사람이 모두 잘 만들기는 어렵습니다. 그래서 현대의 로봇 소프트웨어는 하나의 거대한 프로그램이 아니라, **여러 개의 작은 프로그램이 서로 데이터를 주고받는 형태**로 만들어집니다. 이 작은 프로그램 하나 하나를 ROS에서는 **노드(node)** 라고 부릅니다.

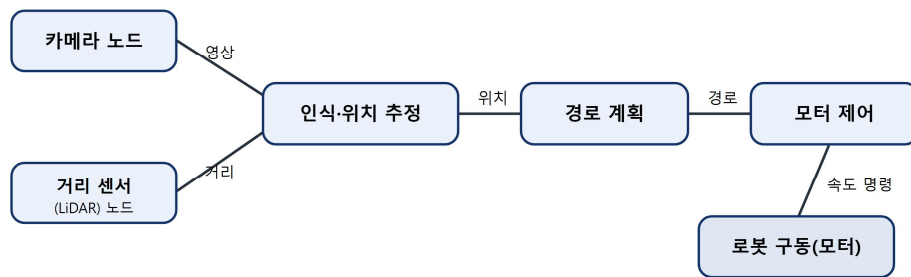


그림 1-1. 카메라·거리센서·경로계획·모터제어 노드가 데이터를 주고받는 로봇 소프트웨어 구조

ROS는 어디에 쓰이나

ROS는 특정 로봇 한 종류만을 위한 기술이 아닙니다. 오늘날 ROS와 ROS2는 우리 주변의 수많은 로봇 속에서 동작하고 있습니다.

- **물류·자율주행**: 창고를 누비는 운반 로봇(AMR), 실내외 배달 로봇
- **제조·협동로봇**: 공장에서 부품을 옮기고 조립하는 로봇 팔(매니퓰레이터)
- **드론·수중 로봇**: 비행 제어와 임무 수행, 해양 탐사
- **서비스 로봇**: 식당 서빙, 병원 물류 이송, 안내 로봇
- **연구·교육**: 전 세계 대학과 기업 연구소의 사실상 표준 플랫폼

물류·제조·우주 분야의 여러 기업과 연구기관이 ROS를 기반으로 로봇을 개발합니다. 이 책에서 다루는 기술은 학습용 예제에 그치지 않고 현업에서 그대로 쓰이는 표준입니다.

ROS의 등장 – 로봇 소프트웨어의 공통 골격

여러 노드가 협력하려면 공통의 규칙이 필요합니다. 데이터를 어떤 형식으로 어떻게 주고받을지 정해진 약속이 없다면, 부품을 새로 만들 때마다 연결 방식을 다시 정의해야 합니다. ROS(Robot Operating System)는 이 약속과 도구를 모아 놓은 것입니다.

ROS는 2007년 미국의 윌로우 개라지(Willow Garage)에서 시작되어, 전 세계 연구자와 기업이 함께 키워 온 오픈소스 프로젝트입니다. ROS가 제공하는 것은 크게 세 가지입니다.

- **통신 방식:** 노드들이 데이터를 주고받는 표준 방법(토픽, 서비스, 액션 등)
- **도구:** 데이터를 들여다보고, 시각화하고, 기록하고, 다시 재생하는 유틸리티
- **생태계:** 센서 드라이버, 내비게이션, 시뮬레이터 등 이미 만들어진 수많은 패키지

덕분에 개발자는 바닥부터 만들지 않고, 검증된 부품을 가져와 자신의 로봇에 맞게 조립할 수 있습니다.

ROS는 운영체제가 아니다

이름에 'Operating System'이 들어가지만, ROS는 윈도우나 리눅스 같은 운영체제가 아닙니다. ROS는 운영체제 **위에서** 동작하며, 여러 노드가 협력하도록 돕는 **미들웨어(middleware)** 이자 개발 도구 모음입니다. 운영체제가 프로그램과 하드웨어 사이를 중개하듯, ROS는 로봇을 이루는 노드들 사이를 중개한다고 이해하면 됩니다.

정의(1) 미들웨어

서로 다른 프로그램이나 시스템이 데이터를 주고받을 수 있도록 중간에서 연결해 주는 소프트웨어 계층. ROS2는 로봇 소프트웨어 노드 간 통신을 책임지는 미들웨어다.

1.2 ROS1에서 ROS2로

ROS1의 한계

ROS1은 로봇 연구의 사실상 표준이 될 만큼 성공했지만, 설계상 몇 가지 한계가 있었습니다.

- **중앙 관리자(master) 의존:** ROS1은 **roscore**라는 중앙 관리자가 모든 연결을 중개합니다. 이 관리자가 멈추면 시스템 전체가 멈춥니다.
- **실시간성 부족:** 정해진 시간 안에 반드시 반응해야 하는 제어에는 적합하지 않았습니다.
- **보안 기능 부재:** 통신을 암호화하거나 접근을 제어하는 장치가 없었습니다.
- **다중 로봇·산업 환경의 어려움:** 여러 대의 로봇, 불안정한 네트워크를 견디는 설계가 아니었습니다.

ROS2의 재설계

ROS2는 이러한 한계를 보완하기 위해 기존 코드를 고치는 대신 **처음부터 다시 설계** 되었습니다. 가장 큰 변화는 통신의 토대를 산업 표준 미들웨어인 **DDS(Data Distribution Service)** 위에 올린 것입니다. 이를 통해 중앙 관리자 없이도 노드들이 서로를 찾아 연결되고, 통신 품질을 세밀하게 조절하며, 보안과 실시간 요구까지 끌어안을 수 있게 되었습니다.

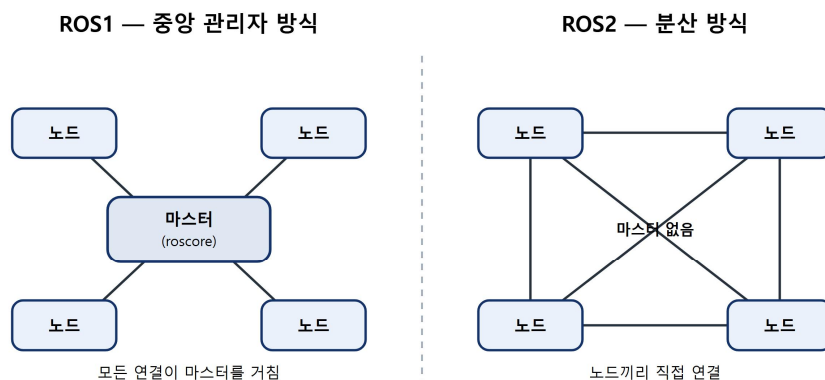


그림 1-2. ROS1(중앙 master 경유) vs ROS2(master 없는 분산 연결) 구조 비교

무엇이 달라졌나

ROS1과 ROS2의 주요 차이를 정리하면 다음과 같습니다.

구분	ROS1	ROS2
통신 기반	자체 프로토콜(TCPROS)	산업 표준 DDS
중앙 관리자	roscore 필요	불필요(분산 발견)
실시간 제어	어려움	지원
보안	없음	지원(SROS2)
빌드 도구	catkin	colcon / ament
Python 클라이언트	rospy	rclpy
C++ 클라이언트	roscpp	rclcpp
주요 대상	연구	연구 + 산업

표 1-1. ROS1과 ROS2의 주요 차이점

표에서 보듯 이름만 바뀐 것이 아니라 토대가 바뀌었습니다. 이 책에서 익히는 **rclpy**(파이썬)와 **rclcpp**(C++)는 ROS2의 표준 클라이언트 라이브러리입니다.

[칼럼] ROS2 버전 이름 이야기

ROS2 배포판은 알파벳 순서로 이름이 붙습니다. Foxy, Galactic, Humble, Iron, Jazzy처럼요. 각 이름에는 거북이(ROS의 상징) 종류나 지명이 담기는데, Jazzy의 정식 이름은 Jazzy Jalisco입니다. 새 배포판은 보통 매년 5월에 발표됩니다.

1.3 ROS2의 심장, DDS

DDS란

DDS(Data Distribution Service) 는 분산 시스템에서 데이터를 안정적으로 주고 받기 위한 국제 표준 통신 규격입니다. 항공, 국방, 금융, 의료처럼 한 치의 오차도 허용되지 않는 분야에서 오래 검증된 기술입니다. ROS2는 이 검증된 기반 위에서 통신을 처리합니다.

마스터가 없다 - 분산 발견

ROS1과 비교했을 때 가장 체감되는 변화는 **중앙 관리자가 사라진** 것입니다. ROS2에서는 각 노드가 네트워크에 자신을 알리고 상대를 스스로 찾아 연결합니다. 이를 **분산 발견(distributed discovery)** 이라고 합니다. 덕분에 시스템 일부가 멈춰도 나머지는 계속 동작하며, 로봇을 켜고 끄는 순서에도 자유로워집니다.

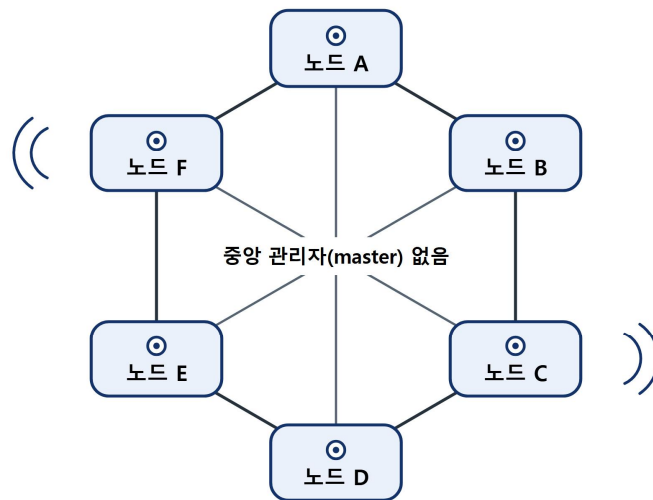


그림 1-3. 노드들이 서로를 직접 찾아 연결되는 DDS 분산 발견 개념

QoS – 통신 품질을 고른다

DDS의 강력한 기능 중 하나는 QoS(Quality of Service, 서비스 품질) 설정입니다. 모든 데이터를 똑같이 보내는 대신, 데이터의 성격에 맞게 전달 방식을 고를 수 있습니다.

- 신뢰성(reliability): 한 건도 빠뜨리지 않고 보낼지(reliable), 좀 놓쳐도 빠른 게 나은지(best effort)
- 이력(history): 최근 몇 개의 데이터를 보관할지
- 지속성(durability): 늦게 접속한 상대방에게도 과거 데이터를 전달할지

예를 들어 카메라 영상은 한두 프레임 놓쳐도 괜찮으니 빠른 전달을, 비상 정지 명령은 절대 놓치면 안 되니 신뢰성 있는 전달을 선택합니다. QoS는 3장에서 직접 다뤄 봅니다.

RMW – DDS를 갈아 끼우는 계층

DDS는 여러 회사가 만든 여러 제품이 있습니다. ROS2는 특정 제품에 묶이지 않도록 RMW(ROS Middleware) 라는 중간 계층을 두어, 코드를 바꾸지 않고도 DDS 구현을 갈아 끼울 수 있게 했습니다. Jazzy의 기본 구현은 eProsima의 Fast DDS이며, 필요하면 다른 구현으로 바꿀 수 있습니다. 이 장 끝의 미니 실습에서

직접 바꿔 봅니다.

정의(2) RMW

ROS2와 DDS 사이를 연결하는 추상화 계층. 덕분에 같은 ROS2 코드가 서로 다른 DDS 제품 위에서 동일하게 동작한다.

ROS2 계층 구조 한눈에 보기

지금까지 ROS2, DDS, RMW를 따로 살펴봤습니다. 이들이 어떻게 층층이 쌓여 동작하는지 한 장의 그림으로 정리해 봅시다. 여러분이 작성한 노드 코드가 맨 위에 있고, 그 아래로 여러 계층을 거쳐 실제 통신을 담당하는 DDS까지 내려갑니다.

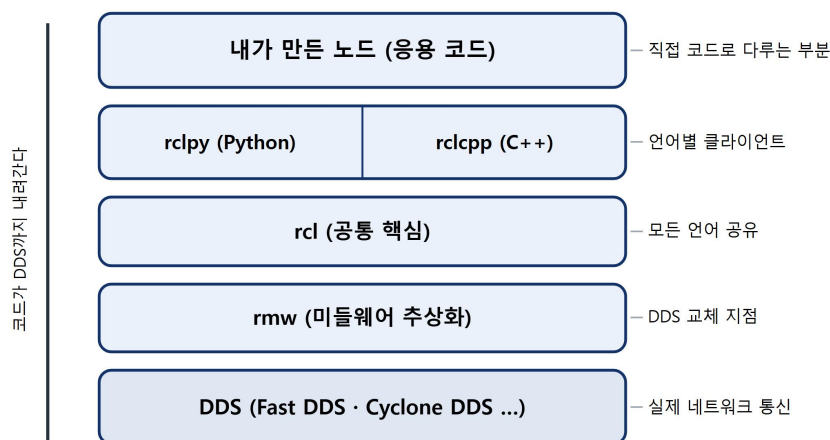


그림 1-4. 응용 코드부터 DDS까지 — ROS2의 계층 구조

위쪽 두 계층(**rclpy/rclcpp**)만 우리가 직접 코드로 다룹니다. 그 아래는 ROS2가 알아서 처리합니다. 여기서 두 가지 중요한 사실이 보입니다. 첫째, **rclpy**와 **rclcpp**가 똑같이 **rcl** 위에 올라가 있기 때문에 **파이썬 노드**와 **C++ 노드**가 아무 문제 없이 통신 할 수 있습니다(1.9절에서 직접 확인합니다). 둘째, **rmw** 계층 덕분에 맨 아래 DDS를 갈아 끼워도 윗부분 코드는 그대로입니다(1.10절 미니 실습에서 확인합니다).

1.4 배포판 선택 — 왜 Jazzy인가

ROS2 배포판과 LTS

ROS2는 대략 1년에 한 번 새로운 배포판(distribution) 을 내놓습니다. 각 배포판에는 알파벳 순서의 이름이 붙습니다. 배포판은 두 종류로 나뉩니다.

- LTS(Long-Term Support, 장기 지원) 배포판: 약 5년간 지원. 안정성이 중요할 때 선택.
- 일반 배포판: 약 1.5년 지원. 최신 기능을 빨리 쓰고 싶을 때 선택.

Jazzy Jalisco

Jazzy Jalisco 는 2024년에 발표된 LTS 배포판으로, 2029년까지 지원됩니다. 짝을 이루는 운영체제는 같은 해 발표된 Ubuntu 24.04 LTS입니다. 새 LTS이면서 운영체제와 지원 기간이 길게 맞물려 있어, 지금 학습을 시작하는 책의 기준 환경으로 가장 적합합니다.

정의(3) LTS

장기 지원 버전. 자주 바뀌지 않고 오랜 기간 보안·버그 수정을 받기 때문에, 교육과 제품 개발의 안정적인 토대가 된다.

이 책의 기준 환경

이 책의 모든 예제는 다음 환경에서 작성하고 검증했습니다.

항목	버전
운영체제	Ubuntu 24.04 LTS (Noble Numbat)
ROS2 배포판	Jazzy Jalisco
기본 DDS	Fast DDS
Python	3.12

표 1-2. 이 책의 기준 개발 환경

버전이 다르면 명령이나 동작이 조금씩 달라질 수 있으니, 학습 단계에서는 같은 환경을 맞추기를 권합니다.

1.5 다섯 가지 통신 미리보기

본격적인 설치에 앞서 앞으로 배울 내용을 개략적으로 살펴봅니다. ROS2에서 노드들은 다섯 가지 방식으로 대화합니다. 각 방식은 2부(3~7장)에서 하나씩 자세

히 다릅니다.

통신 방식	한 줄 설명	비유	학습
토픽(topic)	한 방향으로 계속 흘러보내기	라디오 방송	3장
서비스(service)	요청하고 답을 받기	질문과 대답	4장
액션(action)	오래 걸리는 작업 + 중간 보고	택배 배송 추적	5장
파라미터(parameter)	노드의 설정값 다루기	환경 설정	6장
런치(launch)	여러 노드를 한 번에 실행	시동 스크립트	7장

표 1-3. ROS2의 다섯 가지 통신 방식

각 방식은 쓰임이 다릅니다. 끊임없이 흐르는 센서 데이터는 토픽으로, 한 번 묻고 답을 받는 계산은 서비스로, 오래 걸리는 이동 명령은 진행 상황을 받아 가며 액션으로 보냅니다. 파라미터는 노드의 설정을 바꾸고, 런치는 이 모든 노드를 한 번에 띄웁니다. 여기서는 각 방식의 쓰임을 개략적으로 구분하는 정도면 충분합니다. 자세한 사용법은 해당 장에서 코드로 익힙니다.

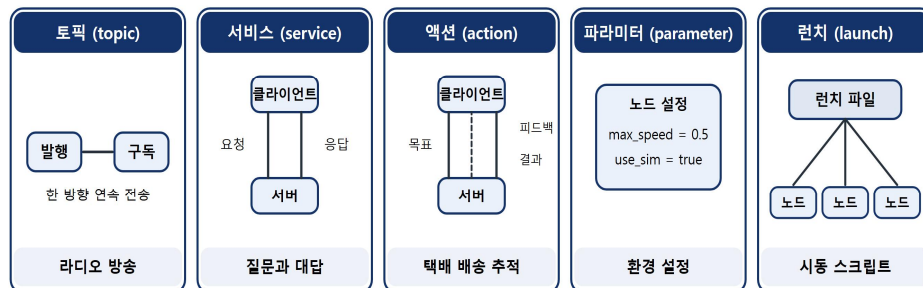


그림 1-5. ROS2의 다섯 가지 통신 방식 한눈에 보기

1.6 개발 환경 준비

Ubuntu 24.04 마련하기

ROS2 Jazzy는 Ubuntu 24.04를 기준으로 합니다. Ubuntu를 준비하는 방법은 여러 가지가 있습니다.

- **네이티브 설치:** PC에 Ubuntu만 설치. 성능이 가장 좋고 권장되는 방식입니다.
- **듀얼 부팅:** 윈도우와 Ubuntu를 함께 설치해 부팅 때 선택.
- **WSL 2:** 윈도우 안에서 리눅스를 실행. 가볍게 시작하기 좋습니다.
- **가상머신:** VirtualBox 등으로 윈도우/맥 위에서 실행. 간편하지만 속도가 느립니다.

처음 학습하는 단계라면 네이티브 설치나 듀얼 부팅을 권하지만, 윈도우를 그대로 쓰면서 시작하고 싶다면 WSL 2가 좋은 선택입니다.

WSL 2로 Ubuntu 설치하기 (윈도우 사용자)

윈도우 11이라면 WSL 2로 Ubuntu 24.04를 손쉽게 설치할 수 있습니다. **관리자 권한 PowerShell** 을 열고 다음을 실행합니다.

```
wsl --install -d Ubuntu-24.04
```

설치가 끝나면 재부팅한 뒤 Ubuntu가 자동으로 실행되며, 사용자 이름과 비밀번호를 한 번 설정합니다. 이후에는 시작 메뉴에서 'Ubuntu 24.04'를 실행하면 리눅스 터미널이 열립니다. 윈도우 11의 WSL 2는 **WSLg** 기능을 내장하고 있어, RViz2나 rqt_graph 같은 그래픽 도구도 추가 설정 없이 창으로 뜹니다. 다만 그래픽 가속이 필요한 무거운 시뮬레이션(2권)은 네이티브 환경을 권장합니다.

정의(4) WSL 2

Windows Subsystem for Linux 2. 윈도우 안에서 실제 리눅스 커널을 구동해 리눅스 프로그램을 그대로 실행하는 기능. 윈도우와 파일을 공유하면서 Ubuntu를 쓸 수 있다.

로케일 설정

설치에 앞서 시스템이 UTF-8 문자를 제대로 다루도록 로케일을 설정합니다. 터미널을 열어 다음을 실행합니다.

```

locale # 현재 설정 확인

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale # UTF-8 적용 확인

```

1.7 ROS2 Jazzy 설치

이제 본격적으로 ROS2를 설치합니다. 설치 순서는 ① 저장소 등록 → ② 패키지 설치 → ③ 개발 도구 설치의 순서로 진행됩니다.

apt 저장소 등록

ROS2는 Ubuntu의 패키지 관리자 **apt**로 설치합니다. 먼저 ROS2 패키지가 들어 있는 저장소를 시스템에 등록해야 합니다. Universe 저장소를 활성화한 뒤, ROS2 저장소 정보를 담은 패키지를 내려받아 설치합니다.

```

# Universe 저장소 활성화
sudo apt install software-properties-common
sudo add-apt-repository universe

# ROS2 저장소 등록 패키지 설치
sudo apt update && sudo apt install curl -y
export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F "tag_name" | awk -F" " '{print $4}')
curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.${ /etc/os-release && echo $VERSION_CODENAME}_all.deb"
sudo apt install /tmp/ros2-apt-source.deb

```

위 과정은 ROS2 저장소의 인증 키와 주소를 시스템에 자동으로 등록해 줍니다.

ROS2 설치

저장소가 등록되었으면 패키지 목록을 갱신하고 ROS2를 설치합니다. 설치 옵션은 여러 가지인데, 학습용으로는 시각화 도구까지 포함된 **desktop** 묶음을 권장합니다.

```
sudo apt update
sudo apt upgrade -y

# 데스크톱 묶음 설치 (RViz, 데모, 튜토리얼 포함)
sudo apt install ros-jazzy-desktop -y
```

ros-jazzy-desktop에는 핵심 라이브러리뿐 아니라 RViz2 같은 시각화 도구와 예제가 함께 들어 있어, 학습에 필요한 대부분이 한 번에 준비됩니다. 설치 용량이 부담된다면 핵심만 담은 **ros-jazzy-ros-base**를 쓸 수도 있지만, 이 책은 **desktop**을 기준으로 합니다.

개발 도구 설치

패키지를 직접 빌드하려면 **colcon**을 비롯한 개발 도구가 필요합니다. 2장부터 사용하니 미리 설치해 둡니다.

```
sudo apt install ros-dev-tools -y
```

1.8 환경 설정

setup 파일 불러오기

ROS2 명령을 쓰려면, 현재 터미널에 ROS2 환경을 불러와야 합니다. 설치 경로의 **setup.bash**를 실행(**source**)하면 됩니다.

```
source /opt/ros/jazzy/setup.bash
```

이제 이 터미널에서는 **ros2** 명령을 쓸 수 있습니다. 잘 되었는지 확인해 봅니다.

```
ros2 --help
```

.bashrc로 자동화하기

문제는 **source** 명령이 현재 터미널에서만 유효하다는 점입니다. 터미널을 새로

열 때마다 매번 입력하기는 번거롭습니다. 터미널이 열릴 때 자동으로 실행되도록 `.bashrc` 파일에 등록해 둡니다.

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

이제부터는 새 터미널을 열면 ROS2 환경이 자동으로 준비됩니다.

정의(5) `source`

셸 스크립트를 현재 터미널 환경에 적용하는 명령. `setup.bash`를 `source`하면 ROS2 명령과 경로가 현재 터미널에 등록된다.

ROS_DOMAIN_ID - 통신 범위를 나눈다

같은 네트워크에 ROS2를 쓰는 여러 사람이 있다면, 서로의 데이터가 섞일 수 있습니다. 분산 발견이 같은 네트워크의 노드를 모두 찾아내기 때문입니다. 이를 막기 위해 `ROS_DOMAIN_ID` 라는 번호로 통신 그룹을 나눕니다. 같은 번호를 쓰는 노드끼리만 서로를 발견합니다.

```
export ROS_DOMAIN_ID=7
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
```

0부터 101 사이의 값을 권장하며, 강의실이나 연구실에서는 사람마다 다른 번호를 쓰면 충돌을 피할 수 있습니다. 혼자 학습할 때는 통신 범위를 자기 컴퓨터 안으로 한정하는 설정도 유용합니다.

```
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
```

RMW_IMPLEMENTATION - DDS 구현 선택

어떤 DDS 구현을 쓸지는 `RMW_IMPLEMENTATION` 환경 변수로 정합니다. 설정하지 않으면 Jazzy의 기본값인 Fast DDS(`rmw_fastrtps_cpp`)가 쓰입니다. 현재 어떤 구현이 동작 중인지는 다음으로 확인합니다.

```
ros2 doctor --report | grep middleware
```

DDS 구현을 직접 바꿔 보는 실습은 1.10절에서 진행합니다. 한 가지만 기억하세요. 서로 통신하는 노드는 같은 RMW 구현을 써야 합니다.

1.9 설치 확인 — 첫 통신

설치가 제대로 되었는지, ROS2가 기본으로 제공하는 예제로 확인해 봅니다. 한 노드가 메시지를 보내고(talker), 다른 노드가 받는(listener) 가장 단순한 통신입니다.

talker와 listener 실행

터미널 1 을 열고 메시지를 보내는 노드를 실행합니다.

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

다음과 같이 0.5초마다 메시지를 보내는 출력이 보이면 정상입니다.

```
[INFO] [talker]: Publishing: 'Hello World: 1'
[INFO] [talker]: Publishing: 'Hello World: 2'
[INFO] [talker]: Publishing: 'Hello World: 3'
```

이제 터미널 2 를 새로 열고 메시지를 받는 노드를 실행합니다.

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

talker가 보낸 메시지를 listener가 받아 출력하면 통신이 성공한 것입니다.

```
[INFO] [listener]: I heard: [Hello World: 10]
[INFO] [listener]: I heard: [Hello World: 11]
```

여기서 talker는 C++(demo_nodes_cpp), listener는 Python(demo_nodes_py)으로 작성된 노드입니다. 서로 다른 언어로 만든 두 노드가 문제없이 통신합니다. 1.3 절에서 본 계층 구조에서 두 노드가 같은 rcl 위에 올라가 있기 때문입니다.

ros2 명령으로 들여다보기

통신이 오가는 동안, 터미널 3 을 열어 시스템을 들여다봅니다. 먼저 실행 중인 노드의 목록을 봅니다.

```
ros2 node list
```

```
/talker  
/listener
```

두 노드가 어떤 토픽으로 대화하는지도 확인할 수 있습니다.

```
ros2 topic list
```

```
/chatter  
/parameter_events  
/rosout
```

`/chatter`가 talker와 listener가 메시지를 주고받는 토픽입니다. 그 내용을 직접 엿볼 수도 있습니다.

```
ros2 topic echo /chatter
```

`ros2 topic echo`는 해당 토픽에 흐르는 데이터를 화면에 그대로 보여 줍니다. 우리가 만들지 않은 통신도 이렇게 들여다볼 수 있다는 것은, 디버깅에 매우 강력한 도구가 됩니다.

`rqt_graph`로 그림으로 보기

명령어 대신 그림으로 보고 싶다면 `rqt_graph`를 실행합니다.

```
rqt_graph
```

talker 노드에서 `/chatter` 토픽을 거쳐 listener 노드로 화살표가 이어진 그림이 나타납니다. 노드와 토픽의 관계를 한눈에 보여 주므로, 시스템이 복잡해질수록 유용합니다.

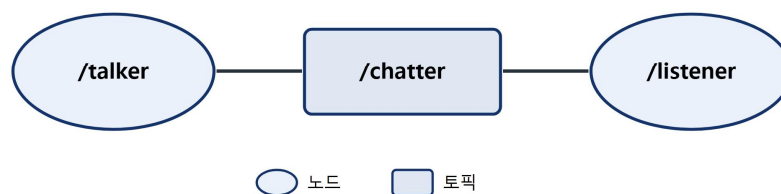


그림 1-6. `rqt_graph`로 본 `talker` → `/chatter` → `listener` 통신 구조

확인이 끝났으면 각 터미널에서 **Ctrl + C**를 눌러 노드를 종료합니다.

1.10 [미니 실습] DDS 구현 바꿔 보기

1.3절에서 ROS2는 RMW 계층 덕분에 DDS 구현을 갈아 끼울 수 있다고 했습니다. 정말 그런지 직접 확인해 봅시다. 기본 Fast DDS 대신 또 다른 대표 구현인 Cyclone DDS 로 바꿔도, 우리가 코드를 한 줄도 고치지 않고 talker/listener가 그대로 동작하는 것을 볼 것입니다.

1단계. Cyclone DDS 구현을 설치합니다.

```
sudo apt install ros-jazzy-rmw-cyclonedds-cpp -y
```

2단계. 두 개의 터미널 모두 에서 RMW 구현을 Cyclone DDS로 지정합니다. 앞서 강조했듯 통신하는 양쪽이 같은 구현을 써야 합니다.

```
# 터미널 1, 터미널 2 모두에서 실행
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
source /opt/ros/jazzy/setup.bash
```

3단계. 적용되었는지 확인합니다.

```
ros2 doctor --report | grep middleware
```

rmw_cyclonedds_cpp가 보이면 성공입니다.

4단계. 다시 talker와 listener를 실행합니다.

```
# 터미널 1
ros2 run demo_nodes_cpp talker
# 터미널 2
ros2 run demo_nodes_py listener
```

기본 Fast DDS일 때와 똑같이 통신됩니다. 같은 코드가 서로 다른 DDS 위에서 동일하게 동작한다는 것, 곧 RMW 계층이 제공하는 이식성을 직접 확인한 결과입니다. 실습을 마쳤으면 `export RMW_IMPLEMENTATION=` 로 변수를 비우거나 터미널을 새로 열어 기본값으로 돌아갑니다.

생각해 보기: 한쪽 터미널만 Cyclone DDS로 바꾸고 다른 쪽은 기본값(Fast DDS)으로

두면 어떻게 될까요? 직접 해 보고 결과를 관찰해 보세요.

1.11 자주 만나는 오류와 해결

설치와 첫 실행에서 자주 마주치는 문제와 해결책을 정리했습니다.

증상	원인	해결
ros2: command not found	setup.bash를 source하지 않음	source /opt/ros/jazzy/setup.bash 실행, .bashrc에 등록
listener가 메시지를 못 받음	두 터미널의 ROS_DOMAIN_ID가 다름	양쪽 ID를 같게 맞춤
RMW 바꾼 뒤 통신 안 됨	한쪽만 RMW_IMPLEMENTATION 변경	양쪽 터미널을 같은 구현으로 맞춤
다른 사람의 노드가 섞여 보임	같은 네트워크·같은 도메인 사용	ROS_DOMAIN_ID를 고유 번호로 변경
ros-jazzy-desktop 설치 실패	저장소 미등록 또는 apt 미갱신	저장소 등록 후 sudo apt update 재실행
add-apt-repository 명령 없음	software-properties-common 미설치	해당 패키지 먼저 설치
WSL에서 rqt 창이 안 뜸	구버전 WSL(WSLg 미지원)	wsl --update로 최신화 후 재시도

표 1-4. 설치·실행에서 자주 만나는 오류와 해결

대부분의 초기 문제는 환경을 source하지 않았거나, ROS_DOMAIN_ID·RMW 구현이 양쪽에서 맞지 않아 생깁니다. 통신이 안 될 때는 이 세 가지를 가장 먼저 의심하세요.

정리

- ROS는 로봇을 이루는 여러 노드가 표준 방식으로 통신하도록 돕는 미들웨어이자 도구 모음이며, 물류·제조·서비스·연구 등 현업 전반에서 쓰입니다.
- ROS2는 ROS1의 한계를 넘기 위해 산업 표준 DDS 위에서 다시 설계되었으며, 중앙 관리자 없이 분산 발견으로 동작합니다.
- ROS2는 응용 코드부터 DDS까지 여러 계층으로 이루어지며, 공통 계층 rcl 덕

분에 파이썬·C++ 노드가 함께 통신하고, **rmw** 덕분에 DDS를 갈아 끼울 수 있습니다.

- 노드는 토픽·서비스·액션·파라미터·런치 다섯 가지 방식으로 대화하며, 이는 2부에서 차례로 배웁니다.

- 이 책은 LTS 배포판인 Jazzy와 Ubuntu 24.04를 기준으로 하며, 윈도우 사용자는 WSL 2로도 시작할 수 있습니다.

- 설치 후에는 `setup.bash`를 `source`하고 `ROS_DOMAIN_ID`를 설정해야 하며, `talker/listener` 예제로 동작을 검증합니다.

핵심 용어

- **ROS(Robot Operating System)**: 로봇 소프트웨어 노드 간 통신과 개발을 돕는 미들웨어·도구 모음

- **노드(node)**: 하나의 일을 맡아 실행되는 ROS2 프로그램의 기본 단위

- **미들웨어**: 서로 다른 프로그램을 중간에서 연결해 주는 소프트웨어 계층

- **DDS(Data Distribution Service)**: ROS2 통신의 기반이 되는 국제 표준 통신 규격

- **분산 발견**: 중앙 관리자 없이 노드가 서로를 스스로 찾아 연결하는 방식

- **QoS**: 통신의 신뢰성·이력·지속성 등 전달 품질을 정하는 설정

- **RMW**: ROS2와 DDS 구현 사이를 잇는 추상화 계층

- **rcl / rclpy / rclcpp**: ROS2의 공통 핵심 계층(rcl)과 그 위의 파이썬·C++ 클라이언트 라이브러리

- **LTS**: 약 5년간 지원되는 장기 지원 배포판

- **ROS_DOMAIN_ID**: 같은 번호끼리만 통신하도록 그룹을 나누는 환경 변수

- **RMW_IMPLEMENTATION**: 사용할 DDS 구현을 지정하는 환경 변수

연습문제

1. ROS가 운영체제가 아니라 미들웨어라고 불리는 이유를 한 문장으로 설명하

라.

2. ROS1의 **roscore**에 해당하는 중앙 관리자가 ROS2에는 없다. 이를 가능하게 한 DDS의 기능은 무엇인가?

3. 파이썬으로 만든 노드와 C++로 만든 노드가 서로 통신할 수 있는 이유를 ROS2 계층 구조(rcl)와 연결지어 설명하라.

4. 다음 상황에 가장 적합한 통신 방식(토픽/서비스/액션)을 고르고 그 이유를 서술하라. (가) 라이다 거리 데이터를 계속 받기 (나) 두 수를 더한 결과를 한 번 받기 (다) 목적지까지 이동시키며 남은 거리를 보고받기

5. 같은 네트워크의 동료와 노드가 섞이지 않게 하려면 어떤 환경 변수를 어떻게 설정해야 하는가?

6. (심화) 1.10절 실습에서 한쪽 터미널만 RMW 구현을 바꾸면 통신이 되지 않습니다. 그 이유를 RMW와 DDS 개념으로 설명하라.